Introduction & Setup

Week 1 · Foundations module · Textbook Chapter 1

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

Friday, August 21, 2026

This week at a glance

Welcome to Web Data Science. The semester starts Thursday, so this week we meet **Friday only**

Today is orientation and introduction to GitHub.

Friday | Orientation, the book & setting up GitHub

- What web data science is and why it matters
- How the class works and how you're graded
- Our unusual textbook
- Git & GitHub

The **Monday** → **Wednesday** → **Friday** rhythm begins next week.

Our textbook

`https://cuinfoscience.github.io/Web-Data-Science-Book/`

Its source code

`https://github.com/cuinfoscience/Web-Data-Science-Book`

Before Monday

Work through `handouts/week-01/setup.md`: Anaconda, Git, and a **free** GitHub account. Bring a laptop — today included.

1. Welcome

About me

Brian C. Keegan — Associate Professor

- Born in Medford, Oregon
- Grew up outside Las Vegas, Nevada
- Mechanical Engineering and Science, Technology & Society, MIT, 2002–06
- Bartender and oral historian
- Ph.D. in Media, Technology & Society, Northwestern, 2007–2012
- Postdoc in computational social science, Northeastern, 2012–14
- Data scientist, Harvard Business School, 2014–16
- CU Boulder Information Science, 2016–present
- Visiting scientist, Harvard Population Center, 2025–26

`https://www.brianckeegan.com`

What I study

High-tempo online collaborations

Public interest data science

Platform governance and migration

Wikipedia, Reddit, Twitter, Bluesky

Ecofascist data science

Cringe-maxxing

I apologize in advance for my geriatric Millennial memes from cooking my brain online

About you

Share out:

- Name and pronouns
- Major and year
- An interesting website normies don't know
- A favorite memory from the summer

What is web data science?

The World Wide Web is the largest, most dynamic, and most contested data source ever created

The Web and the Internet are two distinct things!

The web is a data source and web data science is how you get, clean, and use it

Web data science is the practice of retrieving data from the web, parsing it into structured formats, and analyzing it to produce knowledge.

Not your usual dataset

The web has data but is not a database:

- HTML tables buried in ad tracker slop
- JSON from rate-limited APIs
- scanned PDFs from a FOIA request
- pages that might vanish tomorrow

Data in the public interest

Many disciplines and professions have developed their own local dialects of web data science: computational social science, data journalism, and civic technology

Data is power and that power that should serve the public.

Who gets to access and benefit from web data is an extremely live issue right now: surveillance, AI, copyright

Ask me about CUPIDS Lab

`https://www.cupids-lab.org/`

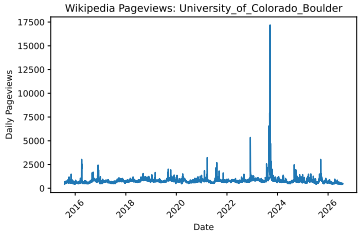
Common sequence across topics

Every week in this course follows a common sequence:

retrieve → **parse** → **analyze**

- **Retrieve** an HTML, JSON, or PDF file with `requests`
- **Parse** it into data: JSON, CSV, *etc.*
- Load into a **pandas** DataFrame
- **Analyze** or visualize the result

You can collect the data and make this visualization after reading the Ch. 01 of the textbook, "Your First Request."



The data science mindset

Technical fluency is not enough. Four dispositions will serve you all semester:

- **Growth mindset —** ability develops through effort. Treat each error as a learning signal, not evidence of inadequacy.
- **Computational thinking —** decompose, recognize patterns, abstract, and specify unambiguous steps.
- **The hacker ethic —** sharing, openness, curiosity, and a bias toward action. The libraries we will use exist because people shared their work.
- **Scientific norms —** communalism, skepticism, and reproducibility. Document your methods; question yours and others' data; report limitations.

2. How this class works

The weekly rhythm

Starting next week, every week runs on the same three-beat rhythm:

Monday | Concept + notebook intro

- Lecture — Introduce the week's ideas and the companion notebook, you read the week's textbook chapter

Wednesday | Notebook lab

- Hands-on — we work and debug the chapter's code together, live

Friday | Textbook revisions

- Fix — submit issues and propose pull requests that improve the textbook and notebooks

---

This week is the exception — Friday only. The full rhythm begins next week.

Evaluation

Your grade is **what you build**, not what you memorize:

- **Notebook Labs — 30%**
  weekly, hands-on data-collection work
- **Textbook Revisions — 15%**
  pull requests + peer reviews that improve the book
- **Attendance — 15%**
  showing up, participating, daily notes
- **Final Project — 40%**
  a research design + real data collection + analysis

No exams

No midterm and no final. You demonstrate learning by **doing the work of a web data scientist** by collecting data, contributing to a shared resource, and building your own project.

Course outline

Module	Week	Dates	Topics
Foundations	1	Aug. 21	Introduction & setup
	2	Aug. 24–28	Ethics, law & responsible collection
	3	Aug. 31–Sep. 4	The post-API age
	4	Sep. 9–11	Data formats: XML & JSON
	5	Sep. 14–18	Web architecture & protocols
Documents	6	Sep. 21–25	Parsing static web pages
	7	Sep. 28–Oct. 2	Archived web pages
	8	Oct. 5–9	Dynamic web pages
	9	Oct. 12–16	PDFs
APIs	10	Oct. 19–23	Wikipedia APIs
	11	Oct. 26–30	Government data APIs
	12	Nov. 2–6	Social media APIs
	13	Nov. 9–13	AI APIs
Practice	14	Nov. 16–20	Automating data collection
	15	Nov. 23–27	**No class: Fall Break**
	16	Nov. 30–Dec. 4	Final presentations
		Final Project due Monday, December 7	

Documentation and testing

Your previous classes may have discouraged online resources.

The training wheels are off.

Finding, reading, and applying documentation is a core professional skill.

Bookmark the docs for the common packages: `requests`, `BeautifulSoup`, `pandas`, `matplotlib`, *etc*.

"It's not working" → say what you **tried**, what you **expected**, what **happened**, where you **looked** for help.

Always credit your sources

Used a blog post, a Q&A answer, docs, or an AI tool to solve something beyond class?

Just include a link in your code.
Undocumented advanced tricks are a reliable signal of uncredited help.

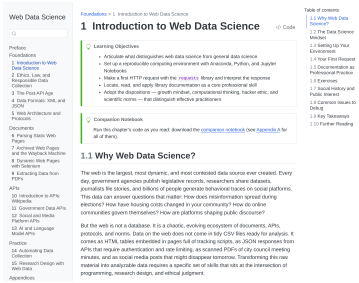
3. The textbook

A book built for this course

There is no textbook to buy — we are writing our own:

- One chapter per week
- Every chapter ships with a companion notebook
- Notebook have exercises we will work on Wednesdays and you will submit on Fridays
- Notebooks might have errors for you to fix or that the repo needs to fix
- We will practice documenting errors as issues and fixing them with pull requests on Fridays

`https://cuinfoscience.github.io/Web-Data-Science-Book/`



How this book was written

Claude Opus and Fable wrote this book using my previous course materials (slides & notebooks) as a foundation

There is important stuff that is wrong or confusing that needs to be fixed

We are going to debug, review, edit, verify it together

This will not be your last time being asked to clean up "AI slop"

- **Confident text that is subtly wrong —** a selector that never matched, a parameter that doesn't exist
- **Uninterpretable explanations —** correct, but skipping where you actually get stuck
- **Decaying examples —** the web changes faster than a book updated once a semester

Why this matters

As new web data scientists, you are the ideal reviewers

Learning how to collaborate with modern development stack of version control, collaboration, and documentation

Remaining vigilant and critical of AI-generated content — especially in academic settings

The book: built and published

The book is a Quarto project built by Claude Fable and Opus agents living in a GitHub repository.

1. Each chapter is a plain-text `.qmd` file — Markdown with executable code blocks
2. GitHub runs `quarto render` to turn the `.qmd` files into HTML files
3. GitHub publishes the HTML result at `https://cuinfoscience.github.io/Web-Data-Science-Book/`
4. Repository has other important files like `references.bib`, `claude.md`, and others we will discuss

4. Git & GitHub

Git is open-source software developed to manage the development of the Linux operating system

Git is **version control** software. It records the history of a project: who changed what, when, and why. It also lets many people work on the project at the same time. Their changes do not overwrite each other.

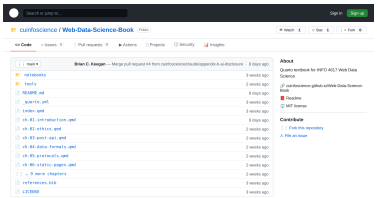
GitHub is a proprietary and popular site for hosting Git projects online and adds a richer social layer of proposing, discussing, and reviewing changes.

Our class materials lives at:

`https://github.com/cuinfoscience/INFO4617-Fall2026`

Our class textbook lives at:

`https://github.com/cuinfoscience/Web-Data-Science-Book`



The book is a living, open repository.

Make or log in to your GitHub account

Create a free account at `github.com` with your `colorado.edu` email to unlock student features.

Choose a professional username (please no “bongdominator420”) and treat this like a portfolio

In today’s daily note, include your GitHub username in the “take away” field

Setup instructions are in `handouts/setup.md` — have this working before next Monday.

Four objects you'll use all semester

- **Repository —** the project: every file and its full history.
- **Issue —** a written report: "here is a problem." Costs nothing but a browser.
- **Pull request —** a proposed change: "here is the fix, side by side with the original."
- **Review —** a response to a PR: comment line-by-line, then approve or request changes.

Issue vs. pull request

An **issue** says *something is wrong*. A **pull request** says *here is the correction* — and shows the exact diff.

Issues are cheap and fast; PRs are the real contribution. **Today you'll file an issue.** From Week 2 on, you'll open pull requests.

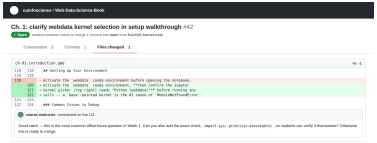
This is how essentially all open-source software gets built — including every library in this course.

Your Friday workflow

From Week 2 onward, every Friday repeats these four steps:

1. **Fork / branch** the book repository
2. **Edit** a chapter's `.qmd` source; commit with a clear message
3. **Open a pull request** describing *what* you changed and *why*
4. **Review** two classmates' PRs — kind, specific, and actionable

A pull request is a **proposal**, not an edit. Your change sits next to the original so peers and I can comment line-by-line before anything merges.



Contributions & reviews are the Textbook Revisions grade (15%). Full workflow walkthrough in Week 2.

5. Activity · Propose a revision

Today's activity: file your first issue

Goal: by the end of class, every one of you has filed one real GitHub issue proposing a change to Chapter 1.

- You need only a **browser** and a **GitHub account**
- Work in pairs; file with individual accounts
- Read **Chapter 1** on the published site — you've just heard the lecture version, so you know what it should have told you

This is a genuine contribution to a publicly-available resource, not a make-work exercise.

Where did the chapter fail you?

That question — not "what could be added in principle" — starts every revision. Sort what you noticed into one of three families:

Broken

Something isn't working

Show: what you ran, and what happened instead.

Unclear

You couldn't follow something.

Show: where you got stuck, and why.

Missing

There's a gap.

Show: what you're expecting.

You are qualified because you're newbies

Nobody who already knows this material can tell me where a first-time reader falls off.

Seven kinds of revision

Not exhaustive or exclusive, just a place to start

#	Type	You noticed…	Your contribution includes	Size
1	**Code drift**	An example errors or returns nothing	The error, the fix, what changed upstream	M
2	**Stale figure**	A screenshot doesn't match today's UI	A fresh screenshot, same filename & crop	S
3	**Common issue**	An error not in "Common Issues to Debug"	Symptom → cause → fix, in the list's format	S
4	**Missing figure**	You sketched it yourself to understand it	The figure, a caption, and alt text	M
5	**Reading**	A claim with no source behind it	The citation in `references.bib` + why it belongs	S
6	**Exercise**	An exercise is ambiguous or unverifiable	Expected output, a rubric, or a starter cell	M
7	**Explanation**	You reread a passage three times	The rewrite + what confused you the first time	M

Grading issues and pull requests

Weekly, out of **10 points**

Criterion	Full marks	Pts
Location	Chapter, section & file named	2
Evidence	I can reproduce what you hit	3
Action	A concrete change, scoped to one thing	3
Reviews	Two peer reviews, specific, with a verdict	2
	Total	**10**

You're graded on the **proposal and the review**, not on whether I merge it.

A well-evidenced PR I decline for editorial reasons earns full credit. A merged one-character typo fix does not.

Little or no credit

- Bulk typo PRs
- "This is confusing"
- Duplicates — check open issues first, then *review* theirs
- Undisclosed AI text you didn't verify

Writing it: the skeleton

Every issue and every PR description uses the same five fields:

- **Title** — `<type>: <specific thing> in Ch. N`
- **Location** — chapter, section, and the `.qmd` file
- **Problem** — what you did, expected, and got. Paste code/output.
- **Why** — who this affects, in one sentence
- **Proposal** — the concrete change you'd make

Do not overload your issues or pull requests with different kinds of fixes.

Worked example

Title: Common issue: kernel/environment mismatch in Ch. 1

Location: `ch-01-introduction.qmd`, "Setting Up Your Environment"

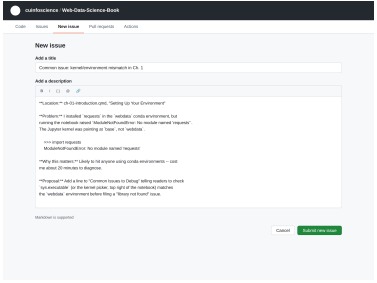
Problem: Installed `requests` in `webdata`, but the notebook raised `ModuleNotFoundError`. The kernel was pointing at base.

Why: Likely to hit anyone using conda environments — cost me 20 minutes.

Proposal: Add to "Common Issues": check `sys.executable` in the notebook.

Your turn

1. **Read** Skim Chapter 1 on the published site (7 min)
2. **Pick one** problem. Which type of problem? (2 min)
3. **Check** if issue already exists, comment instead (1 min)
4. **Write and file** it using the five-field skeleton (5 min)
5. **Share out —** a few of you read yours aloud (5 min)



Stuck choosing? "The chapter never told me X, and I needed X" is always a legitimate issue.

6. Before you go

Before Monday

Complete your daily note **today**:
`https://bit.ly/info4617f26`

Work through `handouts/setup.md` — about 45 minutes:

1. **Python, via Anaconda** — plus a `webdata` environment and libraries
2. **Notebook** — download and run the notebook for Ch. 01
3. **GitHub** — installed and logged in

The handout ends with a two-cell check: retrieve a page, then pull Wikipedia pageviews into a DataFrame. **If it runs, you're ready.**

Don't suffer in silence

Email me if setup fails or you can't access a laptop or install software so we can arrange an accommodation.

Key takeaways

1. Web data science is **retrieve** → **parse** → **analyze**
2. Every week has a rhythm: **Monday** concept, **Wednesday** notebook lab, **Friday** textbook revisions.
3. The textbook is **open, Quarto-built, and AI-drafted —** which is exactly why your first-time-reader perspective is valuable.
4. Revisions to textbook graded on **location, evidence, actionability, and reviews —** not on whether I merge them.
5. Your grade is what you build: Labs 30%, Revisions 15%, Attendance 15%, Final Project 40% **— no exams**.

Next week: **Ethics, law & responsible collection —** `robots.txt`, Terms of Service, and how to collect data without doing harm.